



IoT-Based
Pet Fan Control System
Using PIR and Ultrasonic Sensors

A Project
Presented to the Faculty of the
School of Management and Information Technology
Information Systems Program
De La Salle – College of Saint Benilde
Manila

In Partial Fulfillment
of the Requirements for the subject
ELECTIVE 2:
IOT Development

Submitted By:
Jose Aurelio N. Juico

Dr. Pierre Pauline R. Abesamis
Professor

Dec 6, 2024



TABLE OF CONTENTS

	Page
Title Page	1
Table of Contents	2
A. Introduction	3
B. Project Description	3
C. Scope and Limitation of the Project	5
D. Concept of Operation	7
E. Presentation of the Developed System	10
F. Hardware Components	13
G. Arduino Source Code	14
H. Video Presentation	19
Developer Profile	19



A. INTRODUCTION

The **IoT-Based Pet Fan Control System Using PIR and Ultrasonic Sensors** is an innovative approach to address the inefficiencies in traditional fan control mechanisms. Mechanical timers lack the adaptability to respond to real-time conditions, leading to inefficiencies such as wasted energy when the fan operates without a pet nearby or excessive wear and tear on the fan due to unnecessarily prolonged operation.

Outdoor pets, especially breeds with thick fur, are particularly susceptible to the hot climate of the Philippines. Providing these pets with a ventilated area equipped with a fan helps them cool off during hot weather. However, pets do not always stay in front of the fan, leading to wasted energy and unnecessary wear and tear on the fan. This inefficiency highlights the need for a “smart” solution that ensures the fan operates only when required. This approach conserves energy and extends the fan's lifespan while ensuring comfort for pets when needed.

B. PROJECT DESCRIPTION

This project aims to provide a proof-of-concept and working prototype of a smart pet fan control system using common sensor modules. The concept can then be adapted to the production of related commercial or industrial products. It aims to replace conventional mechanical timers that often unnecessarily waste energy by running fans, even when pets are not present, and contribute to wear and tear due to prolonged operation.



Fig 1. Mechanical timer used to control fan operation

Figure 1 is a photograph of a mechanical timer that could be used in controlling the operational cycle of the fan. Users can select which times the fan (or any other electrical device) can receive power.

The system incorporates Passive Infrared (PIR) and ultrasonic sensors to detect the pet's presence and proximity to automatically control the fan. Users can also control the fan and customize its operation via an Android application. Communication between the hardware and mobile app is facilitated through a Firebase Realtime Database.

This IoT-Based Pet Fan Control System will be able to:

1. Automatically control fan operation depending on the sensor readings.
2. Manually control fan operation through an Android application.
3. Conserve energy and reduce the wear and tear of the fan.



Automated Fan Control:

The system uses Passive Infrared (PIR) and ultrasonic sensors to detect motion and measure proximity. This enables the system to

- Automatically turn the fan on when a pet is detected for a sustained duration of 2 seconds within a defined range of 100 centimeters.
- Keep the fan running as long as the pet remains within the proximity zone and until the set timer expires.
- Automatically turn the fan off when the pet is not detected anymore for the set duration of 2 seconds and within the defined range of 100 centimeters.

Android application for remote control:

The Android application serves as the user interface for remote control and customization of the timer duration. It enables the user to

- Override the automatic detection and timer settings and manually control the fan operation.
- Monitor the status of the fan operation and the remaining timer duration.
- Set the duration of the timer (for demonstration purposes, in seconds) for the fan operation to cycle through.
- Update the fan settings stored on the Firebase Realtime Database.

Intended Users:

1. Pet owners
2. Animal shelters
3. Energy-conscious households
4. Workplaces with frequent worker movement

C. SCOPE AND LIMITATION OF THE PROJECT

Scope:

1. Sensor Integration
Sensors will detect the presence of a pet in the vicinity and dynamically adjust the fan's operation.



2. Hardware Integration

The system will be developed on the ESP32 microcontroller connected to a relay module to control the fan's operation based on sensor inputs.

3. IoT Integration and App Development

The fan's operation can be controlled and customized using an Android application, with communication facilitated through a Firebase Realtime Database (RTDB).

Limitations:

1. Proof of Concept: A mini desk fan will be used for demonstration purposes, and the system may require further adaptation for full-scale household fans.
2. System Performance: The performance of the system will be limited by the functionality and specifications of the sensors as well as the network integrity. Environmental factors may also affect its effective operation.

Assumptions:

1. Proof of Concept
The project is assumed to be only a proof of concept and is not designed as a complete end product.
2. Indoor Use
The device is designed and intended to be used indoors with stable environmental conditions.
3. Limited Performance
The system has limited performance based on the specifications of the components used and the integrity of the network.
4. User Operation
The users are assumed to have a basic understanding of operating the device.



D. CONCEPT OF OPERATION

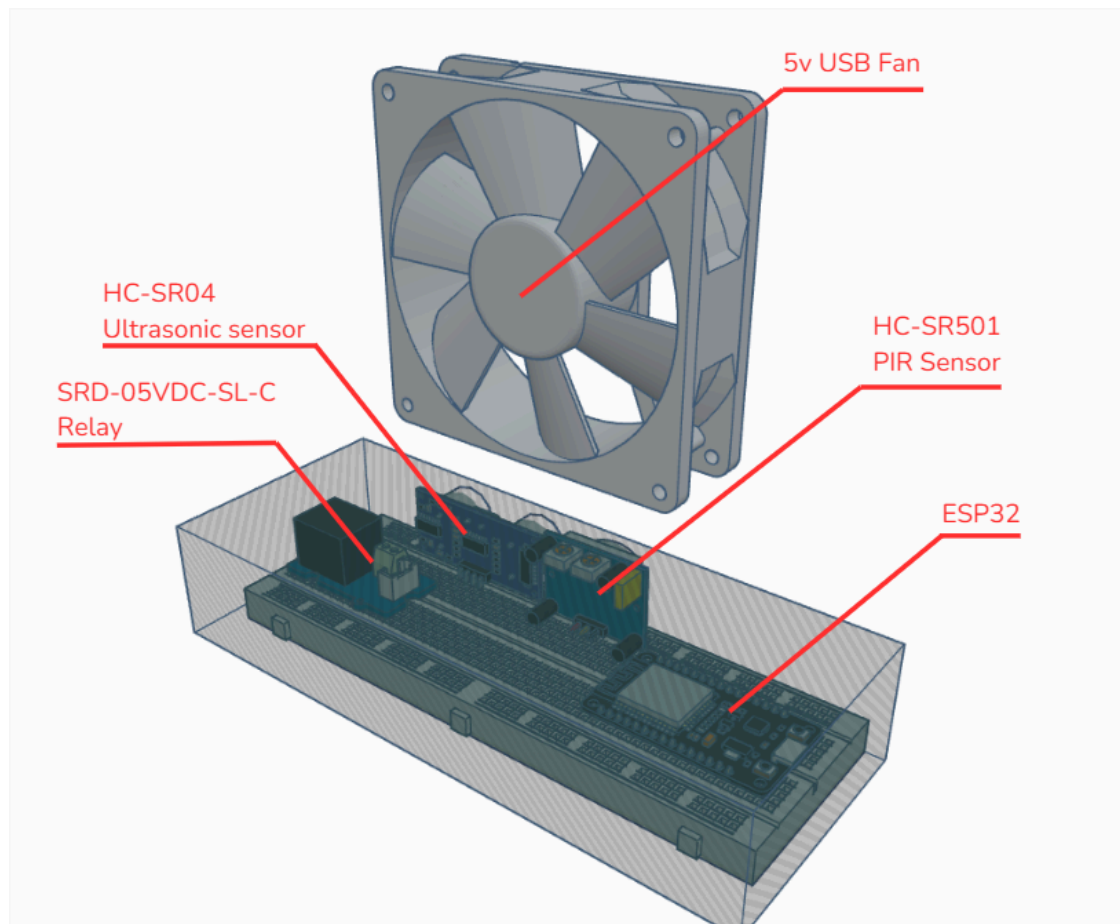


Fig 1. Initial design concept showing the positioning of components

Figure 1 is a 3D model of the device showing the positioning of the components inside the rectangular housing. The PIR and ultrasonic sensors are positioned facing the front for optimal sensor reading. The components will be connected to the ESP32, along with the relay that will allow the fan to be turned ON or OFF. The fan will be placed on top of the housing of the system.

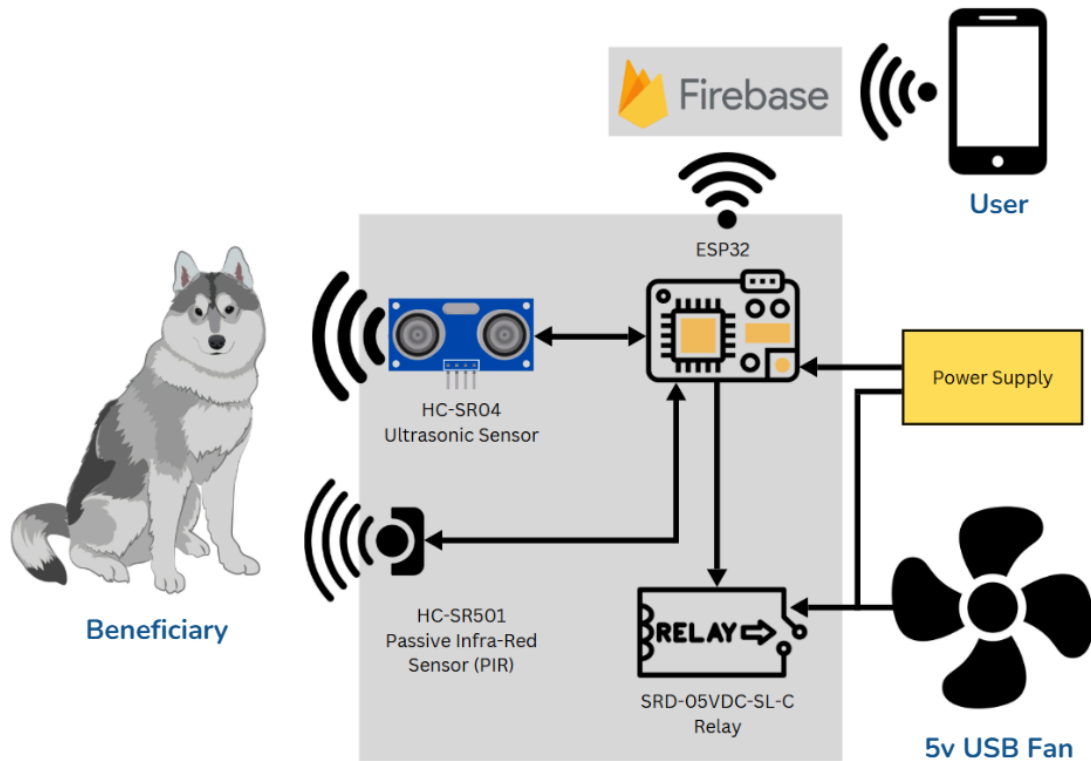


Fig. 2 High-level diagram

Figure 2 shows the high-level diagram of the system including all of the components used and an indication of where the users interact with the system. The power supply is externally located while all the other components (except for the actual fan) are located inside the housing. The pet interacts with the system through the sensors while the pet owner interacts through the Android application.

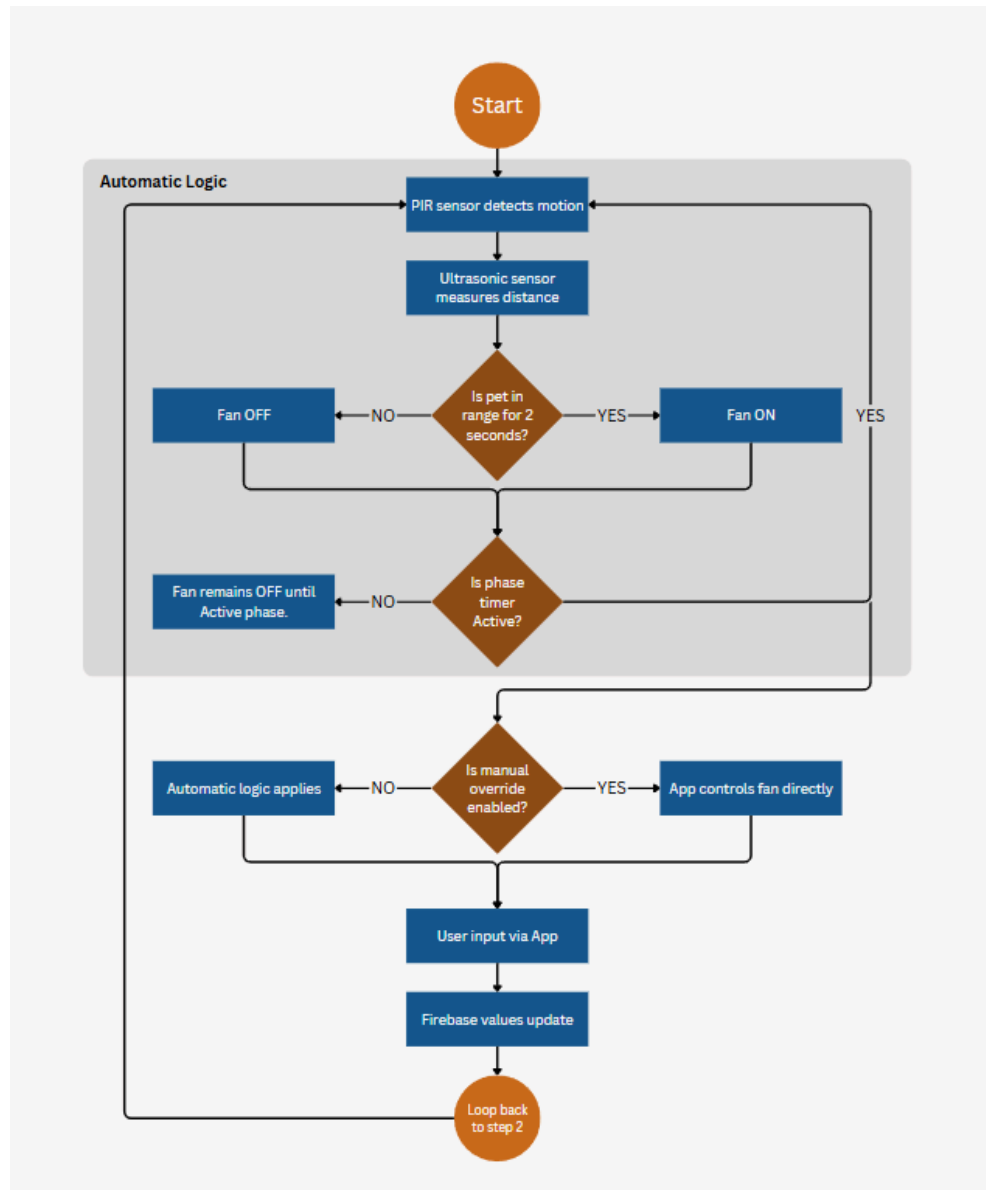


Fig. 3 Process flow diagram

Figure 3 shows the process flow diagram of the system. The process starts when a motion is detected by the PIR sensor. The ultrasonic sensor then checks if there is a sustained detection of the pet within 100 centimeters and for a duration of 2 seconds. After that, it checks for the status of the phase timer, and whether the fan can be turned ON or OFF. The fan turns ON or OFF depending on the status of these checks. The last process to initialize is the updating of the Firebase RTDB values.

E. PRESENTATION OF THE DEVELOPED SYSTEM

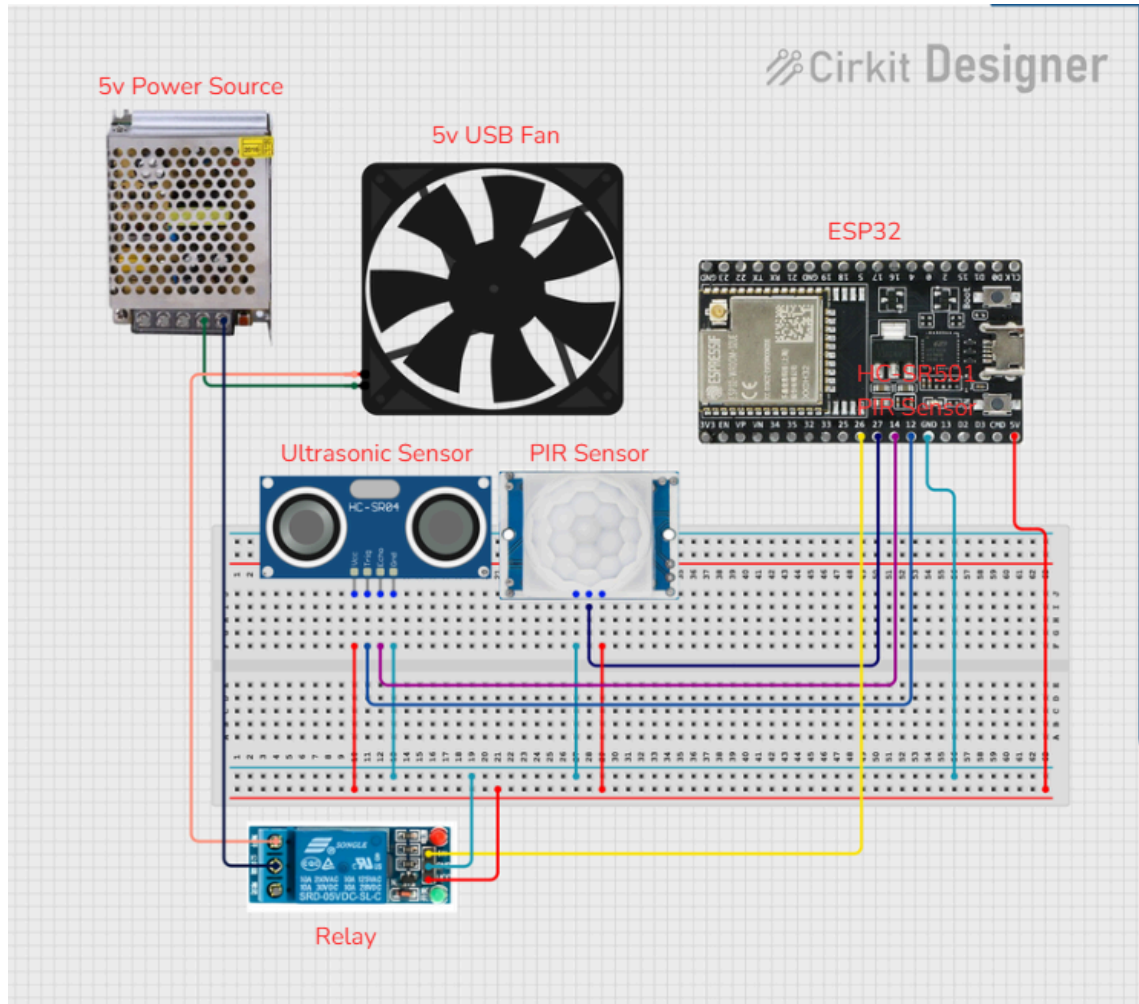


Fig. 4. Circuit diagram of the system

Figure 4 shows the circuit and wiring diagram of the system illustrating how the components are interconnected. The components, ESP32 microcontroller, HC-SR501 PIR sensor, HC-SR04 ultrasonic sensor, and SRD-05VDC-SL-C relay are all connected to each other through a breadboard. Each component's signal pin is wired to a dedicated digital I/O pin on the ESP32. The fan is powered externally by a 5V power source while the ESP32 is powered through its USB port.

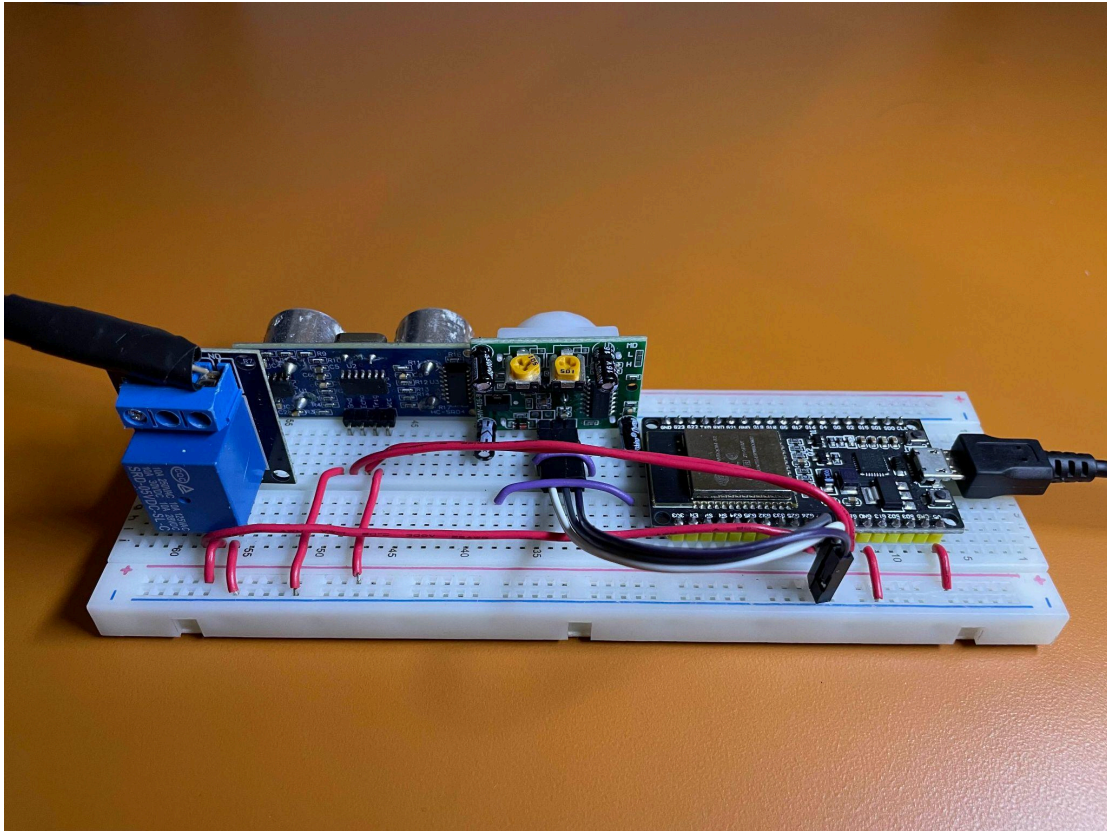


Fig. 5 Completed wiring of the system components

Figure 5 shows a photograph of the completed wiring and component placement of the system. The components pictured are the ESP32 microcontroller, HC-SR501 PIR sensor, HC-SR04 ultrasonic sensor, and SRD-05VDC-SL-C relay. They will then be placed inside a cardboard housing.



Fig. 6. Completed developed system

Figure 6 shows a photograph of the completed developed system with all the components placed inside the cardboard housing. The system will execute its automatic fan control logic when it is powered ON.



Fig. 7. Photograph of me and my pet with the smart fan on automatic operation

Figure 7 shows a demonstration of the smart fan in its intended use case. The fan operates based on the automated logic implemented in the system. In this demonstration, the fan successfully fulfills its purpose of providing the pet with cool air.

F. HARDWARE COMPONENTS

Hardware Components

1. ESP32 microcontroller
The central controller for processing input from the sensors, managing the relay, and communicating with the Firebase Realtime Database.
2. HC-SR501 PIR sensor
Detects motion to trigger the initial activation of the fan.
3. HC-SR04 ultrasonic sensor
Validates the presence of a pet by measuring its distance from the sensor, allowing sustained operation of the fan.
4. SRD-05VDC-SL-C Relay
A switch to control fan operation based on sensor data.
5. USB power supply
5V power source for the system.



G. SOURCE CODE

```
#include <WiFi.h>
#include <Firebase_ESP_Client.h>
#include "addons/TokenHelper.h"
#include "addons/RTDBHelper.h"

#define WIFI_SSID "*****" // Redacted for privacy and security
#define WIFI_PASSWORD "*****"
#define API_KEY "*****"
#define DATABASE_URL "*****"
#define USER_EMAIL "*****"
#define USER_PASSWORD "*****"

#define PIR_PIN 27
#define TRIG_PIN 12
#define ECHO_PIN 14
#define RELAY_PIN 26

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

// Fan and control variables
int pirState = LOW; // State of PIR sensor
float distance = 0; // Distance for ultrasonic sensor
unsigned long phaseTimer = 0; // Timer for current fan phase (Active/Inactive)
unsigned long phaseDuration = 60000; // Default phase duration (milliseconds)
unsigned long noPetTimer = 0; // Timer for no pet detection
unsigned long distanceCheckTimer = 0; // Timer for sustained distance
unsigned long lastManualOverrideUpdate = 0; // Timer for manual override updates
const unsigned long manualOverrideInterval = 1000; // Interval for manual override
bool fanOn = false; // Current fan state
bool fanAllowed = true; // Restriction for turning ON fan
bool pirTriggered = false; // State of motion detection
bool manualOverride = false; // State of manual override mode
bool switchState = false; // State of fan in manual override mode (ON/OFF)
```



```
void setup() {
    Serial.begin(115200);

    pinMode(PIR_PIN, INPUT);
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(RELAY_PIN, OUTPUT);
    digitalWrite(RELAY_PIN, LOW);

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.print(".");
    }
    Serial.println("\nConnected to Wi-Fi");

    config.api_key = API_KEY;
    config.database_url = DATABASE_URL;
    auth.user.email = USER_EMAIL;
    auth.user.password = USER_PASSWORD;
    Firebase.begin(&config, &auth);
    Firebase.reconnectWiFi(true);

    if (Firebase.ready()) {
        Serial.println("Firebase initialized successfully!");
    } else {
        Serial.println("Firebase failed to initialize.");
    }

    phaseTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    updateFirebase();

    // Check if manual override is enabled
    if (manualOverride) {
```



```
if (currentMillis - lastManualOverrideUpdate >= manualOverrideInterval) {
    lastManualOverrideUpdate = currentMillis;
    digitalWrite(RELAY_PIN, switchState ? HIGH : LOW);
    fanOn = switchState;
    Serial.printf("Manual Override: Fan is %s.\n", switchState ? "ON" :
"OFF");
}
return;
}

updatePhase();

// Check if fan can be turned ON
if (!fanAllowed && fanOn) {
    digitalWrite(RELAY_PIN, LOW);
    fanOn = false;
    Serial.println("Fan OFF: Phase restriction active.");
}

pirState = digitalRead(PIR_PIN);

// Check if motion is detected
if (fanAllowed && pirState == HIGH && !pirTriggered) {
    pirTriggered = true;
    distanceCheckTimer = currentMillis;
    Serial.println("Motion detected. Verifying proximity...");
}

// Check if the pet is within distance
if (fanAllowed && pirTriggered && !fanOn) {
    distance = getDistance();
    if (distance > 0 && distance <= 100) { // Range of pet detection
        if (currentMillis - distanceCheckTimer >= 2000) { // Timer for sustained
detection
            digitalWrite(RELAY_PIN, HIGH);
            fanOn = true;
            pirTriggered = false;
            Serial.println("Fan ON: Sustained proximity detected.");
        }
    } else {
```



```
        pirTriggered = false;
        Serial.println("No sustained proximity detected. Resetting.");
    }
}

// Check if no pet is detected for 2 seconds
if (fanAllowed && fanOn) {
    distance = getDistance();
    if (distance > 100 || distance == 0) {
        if (noPetTimer == 0) {
            noPetTimer = currentMillis;
        } else if (currentMillis - noPetTimer >= 2000) {
            digitalWrite(RELAY_PIN, LOW);
            fanOn = false;
            noPetTimer = 0;
            Serial.println("Fan OFF: No pet detected for 2 seconds.");
        }
    } else {
        noPetTimer = 0;
    }
}

// Measures distance of pet (ultrasonic sensor)
float getDistance() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH);
    float distance = duration * 0.034 / 2;

    return distance;
}

// Updates Firebase values
void updateFirebase() {
    if (Firebase.RTDB.getBool(&fbdo, "/ManualOverride")) {
```



```
    manualOverride = fbdo.boolData();
}

if (manualOverride) {
    if (Firebase.RTDB.getBool(&fbdo, "/Switch")) {
        switchState = fbdo.boolData();
    }
}

if (Firebase.RTDB.getInt(&fbdo, "/Timer")) {
    unsigned long newPhaseDuration = fbdo.intData() * 1000;
    if (newPhaseDuration != phaseDuration) {
        phaseDuration = newPhaseDuration;
        phaseTimer = millis();
        fanAllowed = true;
        Firebase.RTDB.setBool(&fbdo, "/PhaseActive", fanAllowed);
        Serial.println("Timer updated and phase restarted.");
    }
}
}

// Updates the current fan phase
void updatePhase() {
    unsigned long currentMillis = millis();

    if (currentMillis - phaseTimer >= phaseDuration) {
        fanAllowed = !fanAllowed;
        Firebase.RTDB.setBool(&fbdo, "/PhaseActive", fanAllowed);
        phaseTimer = currentMillis;

        if (fanAllowed) {
            Serial.println("Phase: Fan can now be turned on.");
        } else {
            Serial.println("Phase: Fan cannot be turned on.");
        }
    }
}

// Countdown for phase timer
unsigned long timeRemaining = (phaseDuration - (currentMillis - phaseTimer))
/ 1000;
```